

Chapter 15

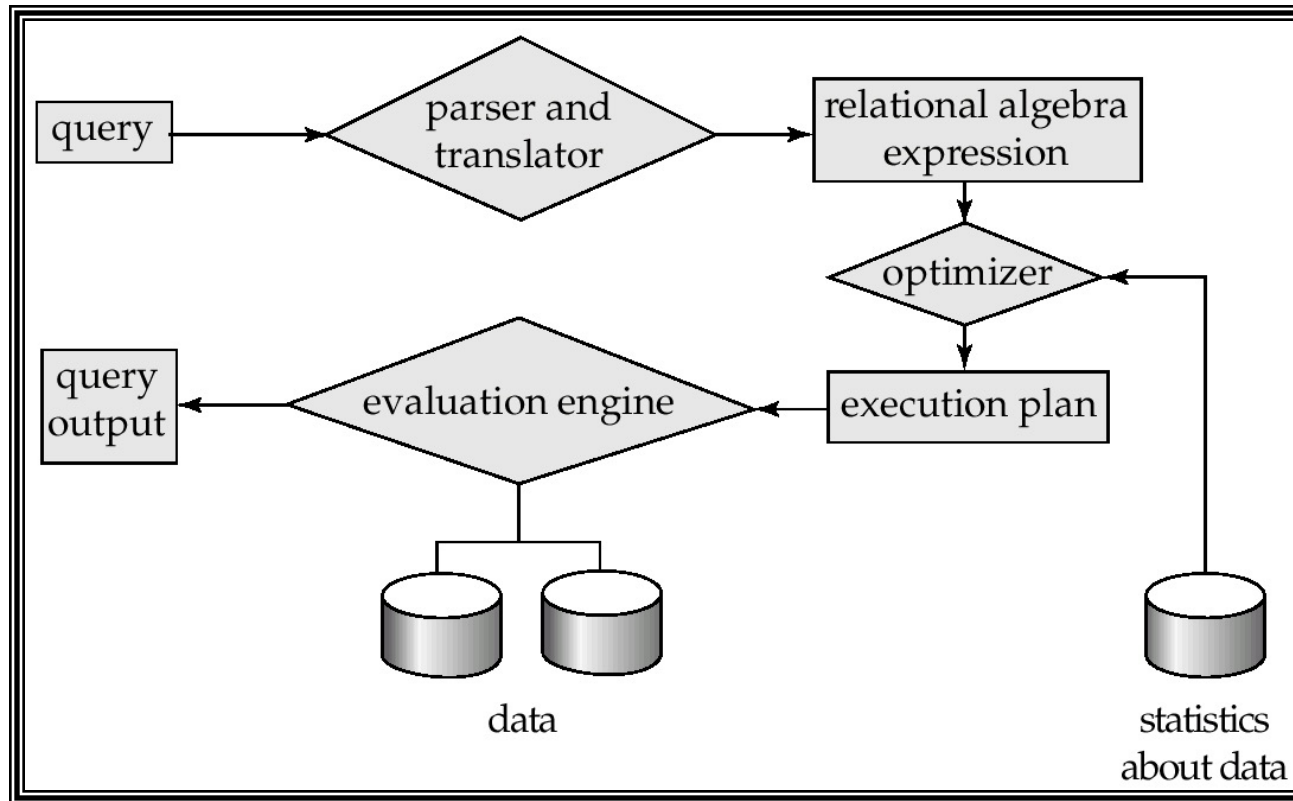
Query Processing



- Basic Steps in Query Processing
- Measures of Query Cost



Basic Steps in Query Processing



1. Parsing and translation
2. Optimization
3. Evaluation



■ Parsing and translation (语法分析与翻译)

★ translate the query into its internal form. This is then translated into relational algebra.

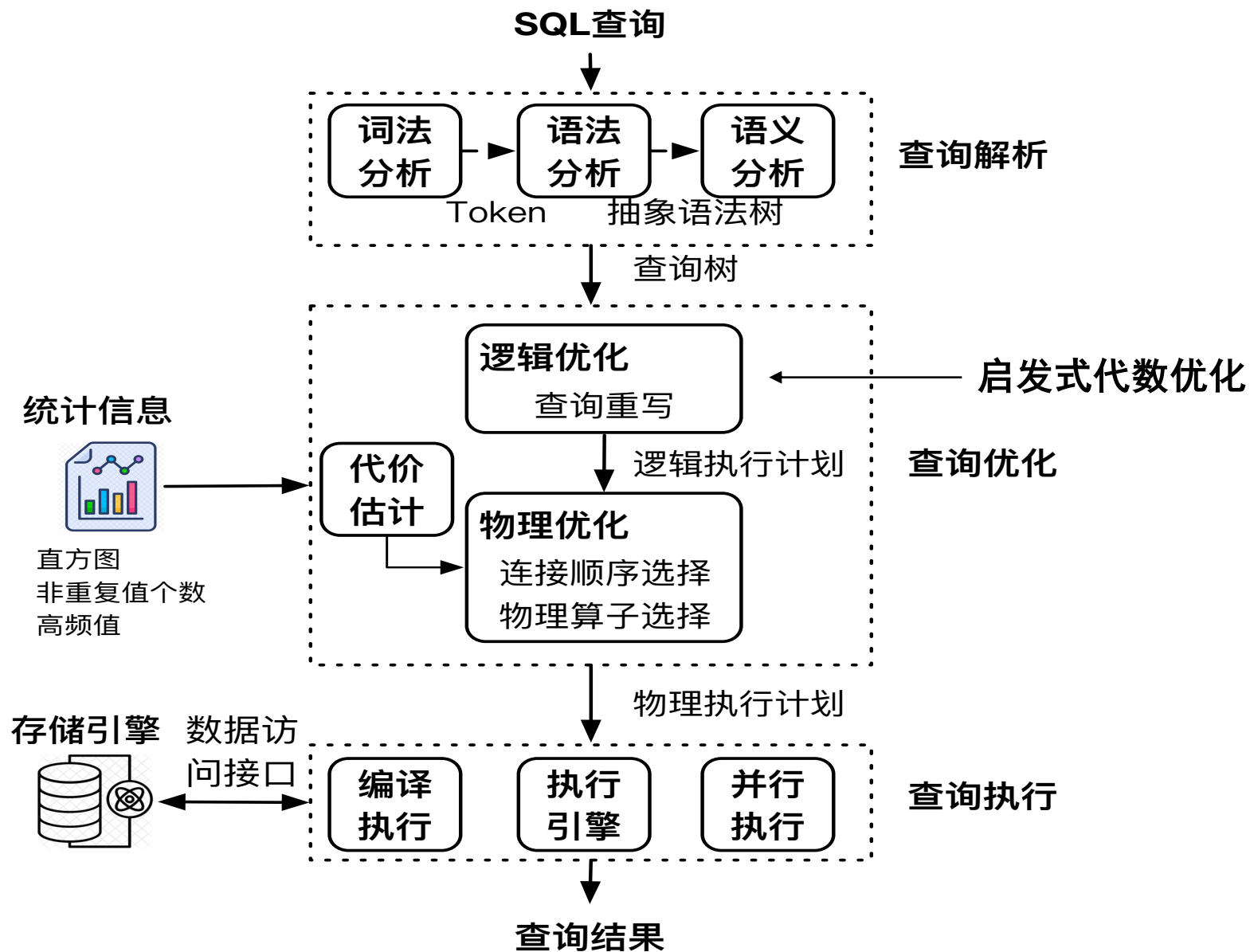
★ Parser checks syntax, verifies relations

■ **Query Optimization (查询优化)** : Amongst all equivalent evaluation plans choose the one with lowest cost.

■ Evaluation (执行)

★ The query-execution engine takes a **query-evaluation plan**, executes that plan, and returns the answers to the query.





- A relational algebra expression may have many equivalent expressions
 - E.g., $\sigma_{salary < 75000}(\Pi_{salary}(instructor))$ is equivalent to $\Pi_{salary}(\sigma_{salary < 75000}(instructor))$
- Each relational algebra operation can be evaluated using one of several different algorithms
 - Correspondingly, a relational-algebra expression can be evaluated in many ways.
- Annotated expression specifying detailed evaluation strategy is called an **evaluation-plan**.
 - E.g., can use an index on *salary* to find instructors with $salary < 75000$,
 - or can perform complete relation scan and discard instructors with $salary \geq 75000$



■ In this chapter

- ★ How to measure query costs
- ★ Algorithms for evaluating relational algebra operations
- ★ How to combine algorithms for individual operations in order to evaluate a complete expression

■ In Next Chapter

- ★ how to optimize queries, that is, how to find an evaluation plan with lowest estimated cost



- Cost is generally measured as total elapsed time for answering query
- Many factors contribute to time cost
 - disk accesses, CPU, or even network communication*
- Typically **disk access** is the predominant cost, and is also relatively easy to estimate. Measured by taking into account
 - Number of seeks * average-seek-cost
 - Number of blocks read * average-block-read-cost
 - Number of blocks written * average-block-write-cost



Measures of Query Cost(2)

■ For simplicity we just use *number of block transfers from disk* and *the number of seeks* as the cost measures

★ ignore CPU costs for simplicity

● t_T – time to transfer one block

● t_S – time for one seek

● Cost for b block transfers plus S seeks

$$b * t_T + S * t_S$$



■ Costs depends on the size of the buffer in main memory

★ Having more memory reduces need for disk access

★ We often use worst case estimates, assuming only the minimum amount of memory needed for the operation is available

■ Real systems take CPU cost into account

■ We do not include cost to writing output to disk in our cost formulae



- **File scan** – search algorithms that locate and retrieve records that fulfill a selection condition.
- Algorithm **A1** (*linear search*). Scan each file block and test all records to see whether they satisfy the selection condition.
 - Cost estimate = $b_r * t_T + t_S$
 - b_r denotes number of blocks containing records from relation r
 - If selection is on a key attribute, can stop on finding record
 - $\text{cost} = (b_r/2) * t_T + t_S$
 - Linear search can be applied regardless of
 - selection condition or
 - ordering of records in the file, or
 - availability of indices



- **A2 (binary search).** Applicable if selection is an equality comparison on the attribute on which file is ordered.
 - Assume that the blocks of a relation are stored contiguously
 - Cost estimate (number of disk blocks to be scanned):
 - cost of locating the first tuple by a binary search on the blocks
 - $\lceil \log_2(b_r) \rceil * (t_T + t_S)$
 - If there are multiple records satisfying selection
 - *Add transfer cost of the* number of blocks containing records that satisfy selection condition



- **Index scan** – search algorithms that use an index
 - selection condition must be on search-key of index.
- **A3** (*primary index on candidate key, equality*). Retrieve a single record that satisfies the corresponding equality condition
 - $Cost = (h_i + 1) * (t_T + t_S)$
- **A4** (*primary index on nonkey, equality*) Retrieve multiple records.
 - Records will be on consecutive blocks
 - Let b = number of blocks containing matching records
 - $Cost = h_i * (t_T + t_S) + t_S + t_T * b$



- **A5** (*equality on search-key of secondary index*).
 - Retrieve a single record if the search-key is a candidate key
 - $Cost = (h_i + 1) * (t_T + t_S)$
 - Retrieve multiple records if search-key is not a candidate key
 - each of n matching records may be on a different block
 - $Cost = (h_i + n) * (t_T + t_S)$
 - Can be very expensive!



- Can implement selections of the form $\sigma_{A \leq V}(r)$ or $\sigma_{A \geq V}(r)$ by using
 - a linear file scan or binary search,
 - or by using indices in the following ways:
- **A6** (*primary index, comparison*). (Relation is sorted on A)
 - For $\sigma_{A \geq V}(r)$ use index to find first tuple $\geq v$ and scan relation sequentially from there
 - For $\sigma_{A \leq V}(r)$ just scan relation sequentially till first tuple $> v$; do not use index



- **A7** (*secondary index, comparison*).

- For $\sigma_{A \geq v}(r)$ use index to find first index entry $\geq v$ and scan index sequentially from there, to find pointers to records.

- For $\sigma_{A \leq v}(r)$ just scan leaf pages of index finding pointers to records, till first entry $> v$

- In either case, retrieve records that are pointed to

- requires an I/O for each record

- Linear file scan may be cheaper



Implementation of Complex Selections DataBase System Concepts

- **Conjunction:** $\sigma_{\theta_1 \wedge \theta_2 \wedge \dots \wedge \theta_n}(r)$
- **A8** (*conjunctive selection using one index*).
 - Select a combination of θ_i and algorithms A1 through A7 that results in the least cost for $\sigma_{\theta_i}(r)$.
 - Test other conditions on tuple after fetching it into memory buffer.
- **A9** (*conjunctive selection using multiple-key index*).
 - Use appropriate composite (multiple-key) index if available.



• **A10** (*conjunctive selection by intersection of identifiers*).

– Requires indices with record pointers.

– Use corresponding index for each condition, and take intersection of all the obtained sets of record pointers.

– Then fetch records from file

– If some conditions do not have appropriate indices, apply test in memory.



- **Disjunction:** $\sigma_{\theta_1 \vee \theta_2 \vee \dots \vee \theta_n}(r)$.
- **A11** (*disjunctive selection by union of identifiers*).
 - Applicable if *all* conditions have available indices.
 - Otherwise use linear scan.
 - Use corresponding index for each condition, and take union of all the obtained sets of record pointers.
 - Then fetch records from file
- **Negation:** $\sigma_{\neg\theta}(r)$
 - Use linear scan on file
 - If very few records satisfy $\neg\theta$, and an index is applicable to θ
 - Find satisfying records using index and fetch from file

